

Interrupts und Ausführungskontexte

Wer unterbricht wen und welche Regeln gelten dann?

3.02

Der Kontext bestimmt die zulässige Kernel-API

PROZESSKONTEXT	Schlafen möglich, aktueller Task vorhanden
WORKQUEUE	Prozesskontext in einem Worker-Task
SOFTIRQ	Atomarer Kontext, kein Schlafen
HARDIRQ	Atomarer Kontext, sehr kurze Bearbeitung
NMI	Stärkste Einschränkungen, Spezialfall

Zuerst den Kontext klären, danach Primitiv und API wählen.

Interrupts unterbrechen den Kontrollfluss

01 Ereignis

Gerät oder Controller setzt einen Interrupt

02 Eintritt

CPU wechselt in den Kernel-Interruptkontext

03 Handler

Treiber bestätigt und erfasst das Ereignis

04 Rückkehr

Unterbrochener Kontext wird fortgesetzt

Ein Interrupt tritt asynchron zum gerade laufenden Code auf.

Der Hard-IRQ Handler erledigt nur das Dringende

HARDIRQ-HANDLER

```
irqreturn_t demo_irq(int irq, void *data)
{
    status = read_status(dev);
    if (!status)
        return IRQ_NONE;

    ack_irq(dev, status);
    queue_work(dev->wq, &dev->work);
    return IRQ_HANDLED;
}
```

IM HANDLER

- Hardware bestätigen
- Daten sichern
- Arbeit anstoßen

NICHT IM HANDLER

- Schlafen
- Blockierendes I/O
- Lange Verarbeitung

Der Handler erfasst das Ereignis und verschiebt aufwendige Arbeit.

Threaded IRQs verlagern Arbeit in einen Task

REGISTRIERUNG

```
request_threaded_irq(irq,  
    demo_irq_top,  
    demo_irq_thread,  
    IRQF_ONESHOT,  
    "demo", dev);  
  
/* top */    return IRQ_WAKE_THREAD;  
/* thread */ handle_event(dev);
```

01 Top Half

Prüfen, bestätigen und den IRQ-Thread wecken.

02 IRQ-Thread

Darf schlafen, Buszugriffe ausführen und priorisiert werden.

IRQF_ONESHOT hält die IRQ-Linie bis zum Thread-Ende maskiert.

Softirqs bearbeiten zeitkritische Folgearbeit

NETZWERK RX/TX

Pakete in begrenzten Budgets verarbeiten

TIMER

Abgelaufene Timer ausführen

RCU

Callbacks nach der Grace Period bearbeiten

KSOFTIRQD

Überlastete Softirq-Arbeit im Kernel-Thread fortsetzen

Softirq-Code bleibt atomar, auch wenn ksoftirqd ihn ausführt.

Tasklets als historische Softirq-Abstraktion

AUSFÜHRUNG

Atomarer Softirq-Kontext

PARALLELITÄT

Dieselbe Instanz läuft nicht gleichzeitig auf zwei CPUs

EINSCHRÄNKUNG

Kein Schlafen, kurze Laufzeit

NEUER CODE

Workqueue oder threaded IRQ bevorzugen

Bestehenden Code verstehen und neue Architektur bewusst wählen.

Workqueues bieten Prozesskontext

WORKER

```
static void demo_work(struct work_struct *work)
{
    mutex_lock(&dev->lock);
    read_device_registers(dev);
    update_state(dev);
    mutex_unlock(&dev->lock);
}

INIT_WORK(&dev->work, demo_work);
queue_work(dev->wq, &dev->work);
```

GEEIGNET

- Längere Verarbeitung
- Schlafende APIs
- Serielle Arbeitsfolgen

BEACHTEN

- Objektlebensdauer
- Flush und Cancel
- Reihenfolge

Workqueues verschieben Arbeit in einen schlafbaren Task-Kontext.

In atomarem Kontext darf kein Pfad schlafen

ZULÄSSIG

spin_lock, atomic, GFP_ATOMIC, queue_work

UNZULÄSSIG

mutex_lock, msleep, GFP_KERNEL, blockierendes I/O

PRÜFHILFE

might_sleep(), Lockdep und Debug-Konfiguration

ENTSCHEIDUNG

Unsichere Arbeit in Thread oder Workqueue verschieben

Ein erfolgreicher Test ersetzt keine Prüfung des Ausführungskontexts.

Preemption und Interrupts sind getrennt

PREEMPT_DISABLE

Kein Taskwechsel lokal, Hardirq bleibt möglich

LOCAL_BH_DISABLE

Lokale Softirqs zurückstellen

LOCAL_IRQ_DISABLE

Lokale Hardirqs deaktivieren

SPIN_LOCK

Preemption lokal aus, andere CPUs serialisiert

SPIN_LOCK_IRQSAVE

Zusätzlich den lokalen IRQ-Zustand sichern

Alle beteiligten Kontexte brauchen Schutz, zusätzliche Sperren jedoch nicht.

irqsave erhält den Zustand des Aufrufers

IRQSAFE-LOCK

```
unsigned long flags;  
  
spin_lock_irqsave(&dev->lock, flags);  
update_shared_state(dev);  
spin_unlock_irqrestore(&dev->lock, flags);
```

01 Warum flags?

Ein Aufrufer kann bereits mit deaktivierten IRQs eintreten.

02 Symmetrie

Dasselbe flags beim passenden Release verwenden.

irqrestore stellt exakt den zuvor gesicherten IRQ-Zustand wieder her.

Sharing bestimmt die passende Lock-Variante

NUR TASKS / CPUS

Mutex oder spin_lock, abhängig von Schlafbarkeit

TASK + SOFTIRQ

spin_lock_bh / spin_unlock_bh

TASK + HARDIRQ

spin_lock_irqsave / spin_unlock_irqrestore

NUR CPU-LOKAL

Per-CPU-Daten plus passende Preemption-Regel

Acquire und Release müssen dieselbe Kontextannahme ausdrücken.

Top und Bottom Half brauchen einen Vertrag

PRODUCER UND CONSUMER

```
/* Hardirq: producer */
spin_lock(&q_lock);
ring[head++] = event;
spin_unlock(&q_lock);
queue_work(wq, &work);

/* Worker: consumer */
spin_lock_irqsave(&q_lock, flags);
event = ring[tail++];
spin_unlock_irqrestore(&q_lock, flags);
```

VERTRAG

- Wer produziert?
- Wer konsumiert?
- Wann ist Speicher gültig?

PRÜFEN

- Queue voll oder leer
- Shutdown
- Cancel und Flush

Übergabe, Synchronisation und Lebensdauer bilden einen Vertrag.

Kontextfehler hinterlassen typische Spuren

SCHEDULING WHILE ATOMIC

Schlafende API im atomaren Kontext

BUG: SLEEPING FUNCTION

might_sleep-Prüfung schlägt an

HARDIRQS-ON/OFF MISMATCH

IRQ-Zustand falsch wiederhergestellt

LOCKDEP SPLAT

Lock-Reihenfolge oder Kontext widersprüchlich

LANGE IRQ-LATENZ

Handler oder Softirq arbeitet zu lange

Erste Warnung vollständig sichern: CPU, Task, IRQ-Status und Stacktrace.

Ausführungskontexte systematisch prüfen

KONTEXT	Prozess, Worker, Softirq, Hardirq oder NMI?
SCHLAFEN	Kann eine aufgerufene API blockieren?
SHARING	Welche Pfade greifen auf denselben Zustand zu?
SYNCHRONISATION	Passt die Lock-Variante zum strengsten Kontext?
LAUFZEIT	Bleibt atomare Arbeit kurz und begrenzt?
LIFECYCLE	Sind Cancel, Flush und Freigabe geordnet?

Kontext zuerst, danach API, Synchronisation und Übergabe.